

# **Design of a Secure Embedded Environmental Sensor Using the B-U585I-IOT02A Discovery Kit**

## **1. Introduction**

Internet of Things (IoT) technology has been spreading widely and environmental sensors deployment has also been largely increased leveraging this technology. Environmental sensors are extensively used for monitoring temperature, humidity, air and water quality and other different environmental parameters. The sensor data collected by these IoT devices may contain proprietary and sensitive information and there is always a possibility of these data being attacked. Attackers might try to intercept these data collection and even alter them or inject some malicious firmware in these devices or gain control of these sensitive devices and hamper the operations somehow. This demands the secure design of sensors. Designers can build a secure system by leveraging the foundation provided by modern microcontrollers, such as, hardware root-of-trust, cryptographic accelerators and isolated execution environments. Based on the B-U585I-IOT02A discovery kit's state-of-the-art security features, this report proposes a complete security architecture for an environmental sensor data collecting sensitive sensor readings.

## **2. Hardware Platform Overview**

### **2.1 B-U585I-IOT02A Discovery kit features**

The B-U585I-IOT02A kit uses STM32U585AI microcontroller chip. This microcontroller unit provides 2MB of flash memory and 786KB of SRAM and integrates 802.11 b/g/n Wi-fi module (MXCHIP) (*B-U585I-IOT02A | Product - STMicroelectronics, n.d.*). It also has a BLE module and MEMS sensors with magnetometer, accelerometer, gyroscope, barometer, humidity and temperature sensor making it suitable for environmental monitoring applications. Furthermore, it also allows other boards like Arduino to connect to it via expansion connectors and includes STLINK-V3E debugger/programmer. All these features of the discovery kit board make the board suitable for utilizing IoT technologies through Wi-Fi or BLE.

STM32U585AI MCU has multiple low-power modes, which allows it for battery-operated operation. Its 32-bit Cortex-M33 core has TrustZone, DSP instructions and memory protection unit. The combination of these features makes the MCU suitable for long-period secure operations in environmental monitoring systems.

## 2.2 Security and Cryptographic capabilities of the board

The B-U585I-IOT02A Discovery kit is integrated with a good set of security features. The MCU is available with Arm TrustZone support with securable I/O, memories and other peripherals. It also has flexible life-cycle scheme along with RDP and the debug is password protected. The hardware root-of-trust (RoT) is enabled by the unique boot entry and secured HDP. Furthermore, STM32U585AI MCU has embedded RSS (Root Secure Services) which facilitates secure firmware installation on the MCU. HUK (Hardware Unique Key) is provided for secure data storage and Trusted Firmware -M helps with secure firmware upgrade. Additionally, it offers dual AES coprocessors, DPA resistant public key accelerator (PKA), a hash accelerator, active tamper detections, a NIST SP800-90B compliant true random number generator (TRNG) and 96-bit unique identifier (*B-U585I-IOT02A Discovery Kit — Zephyr Project Documentation*, n.d.). With all these integrated hardware features, the MCU provides strong security mechanisms and cryptographic foundation.

Some of these features and their importance are summarized in Table 1.

**Table 1:** Summary of key features of STM32U585AI

| Features                                    | Description                                                         | Purpose                                                                                       |
|---------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>TrustZone support with securable I/O</b> | Divides the MCU into domains that are secure and those that are not | Keeps sensitive data and code protected and prevents untrusted code from accessing those info |
| <b>RoT (unique boot entry, HDP)</b>         | Ensures only trusted firmware executes at startup                   | Provides foundation for booting and updating securely                                         |
| <b>Root Secure Services (RSS)</b>           | Helps firmware installation securely                                | Useful for updating firmware in a secure way                                                  |
| <b>Hardware Unique Key (HUK)</b>            | Unique secret per device                                            | Protects encryption keys and enables secure storage                                           |
| <b>AES coprocessors &amp; PKA</b>           | Hardware accelerators for symmetric and public-key cryptography     | Helps in efficiently perform encryption, decryption                                           |
| <b>TRNG</b>                                 | True Random Number Generator                                        | Provides entropy for keys                                                                     |
| <b>Active Tamper detection</b>              | Helps to detect any physical tampering                              | Useful in protecting secrets                                                                  |

### 3. Security Requirements and Threat Model

Identifying the objectives of security and what potential threats may harm the device is an important step in the embedded system design. The environmental sensors are deployed to collect different environmental readings like temperature, humidity, air quality and it transmits these data to a cloud-based service. Also, these devices should be capable of supporting remote firmware updates to upgrade its features/capability. Hence, for this application, the primary security goals include:

- The sensor data and firmware should be protected strongly against any eavesdropping during every storage and transmission processes.
- Any unwanted and unauthorized alteration of device firmware, configuration or sensor data should be strictly prevented.
- Firmware updates should be performed only if it is originated from a trusted authority.
- A robust, failure-resistant Firmware-on-the-air mechanism aligned with best practices and standards (Moran et al., 2021)
- Device functionality should be maintained despite any attacks and it should enable reliable OTA updates and recovery mechanisms.
- Verifiable records of firmware versions should be maintained and should support anti-rollback.

Based on these security goals, the potential threats that might hinder the device and its application needs to be identified and addressed. Some of the potential threats during the application are:

- Firmware attacks: This type of threat includes installation of malicious software and execution of unauthorized code via unsecured boot path.
- Network attacks: Eavesdropping, altering or intercepting of communication on Wi-Fi and BLE channels is also a potential threat to the system.
- Data at rest attacks: Cloning or stealing sensitive credentials and keys and data breach is another threat that might harm the system (“What Is Data at Rest | Security & Encryption Explained | Imperva,” n.d.).
- Physical and debug attacks: Attackers with physical access to the device can exploit the JTAG/SWD to read the firmware or inject unwanted changes to the firmware (*Understanding Physical Security Attacks | Exploiting Local Host and Physical Security Vulnerabilities | Pearson IT Certification*, n.d.). Additionally, fault injection technique can be used to bypass the secure boots.
- Physical Tampering: Since the device environmental sensors may be deployed in some unsecured locations, physical tampering is also one of the potential threats.

## **4. Secure Boot Design and Root of Trust (RoT)**

### **4.1 Implementation of RoT**

Robust RoT is required for starting a system in a secure way. There is ST-Internal RoT (STiRoT) used by STMicroelectronics which is stored in a secured flash area and it provides first boot stage. It helps in booting the system securely and also provide secure firmware update. This is executed at every reset and it makes sure that there is only one entry point at reset (*Category:Secure Boot and Secure Firmware Update - Stm32mcu*, n.d.).

The RoT implements PSA cryptograph with algorithm such as ECDSA, AES and SHA-256 hashing to validate and decrypt the firmware images while performing the secure boot. The strategy used by RoT for making the system more secure involves creating a protected reserve for secrets, disabling the JTAG and debug access to prevent debug attack during boot, monitoring any tampering events and limiting the execution surface during the booting process. All these are collectively implemented by different hardware mechanisms like HDP, active tampers and Trustzone (*Security:STiRoT - Stm32mcu*, n.d.).

### **4.2 Secure Boot Process**

Whenever the MCU is powered on or reset, its secure domain runs STiRoT. It initializes the secure environment for booting by configuring TrustZone and setting up the cryptographics accelerators. Additionally, it verifies whether the firmware in the next stage (TF-M bootloader) is from a trusted source or it has been altered in any unwanted way.

STiRoT implements AES-CTR-128 and device's secret keys to decrypt the firmware into secure memory, if the firmware is in the encrypted form (*Security:STiRoT - Stm32mcu*, n.d.). Furthermore, it provides reliability against rollback by verifying the version metadata in order to avoid the installation of older, vulnerable firmware. After the firmware is validated, depending on the configuration, it transfers the control to the next stage which is either secure bootloader or application code.

Hence, the root of trust ensures that only authentic and upgraded firmware is run on the embedded system by using a secure boot mechanizes and hence any unwanted or rollback software cannot bypass the RoT.

## **5. Secure Firmware Execution and Application Isolation**

### **5.1 TrustZone and Trusted Firmware-M**

The system is divided into two zones: secure world and non-secure world. The secure world is protected and isolated form the non-secure world. Secure world holds sensitive code while the non-secure world is responsible for executing the main application and Arm's TrustZone has the duty of isolating these

two worlds. By the use of securable I/O and peripherals, it is ensured that the sensitive and secure functions in the secure world cannot be accessed from the non-secure world (*B-U585I-IOT02A Discovery Kit — Zephyr Project Documentation*, n.d.). The application assigns memory regions and interrupts for each domain, with hardware enforcement.

Arm's Platform Security Architecture (PSA) is implemented by the Trusted Firmware-M (TF-M) and TF-M is integrated by ST's board support on the STM32U5 MCU. Secure storage, secure bootloader, attestation and cryptography are some of the features provided by TF-M. TF-M also assists in upgrading the firmware securely by downloading new images and then verifying their authenticity as well as integrity. If the firmware checks as the trusted one, then it installs them. Protected storage and internal trusted storage services are also offered by TF-M which rely on AES-GCM encryption as well as hardware isolation in order to protect the data (*Security:STiRoT - Stm32mcu*, n.d.).

## 5.2 Secure and Non-secure world

As stated earlier, the system is divided into secure and non-secure worlds. TF-M core runs inside the secure world and provides following services:

- Cryptographic services: It includes functions like random number generation, encryption and decryption and key storage.
- Protected storage service: It is responsible for storing any configuration data, credentials as well as sensor calibration keys. This utilizes the feature of HUK and other derived keys.
- Attestation service: It provides the identity of device and firmware measurements to cloud-based servers.
- Bootloader service: One of its major jobs is to handle the firmware authentication and securely update the firmware.

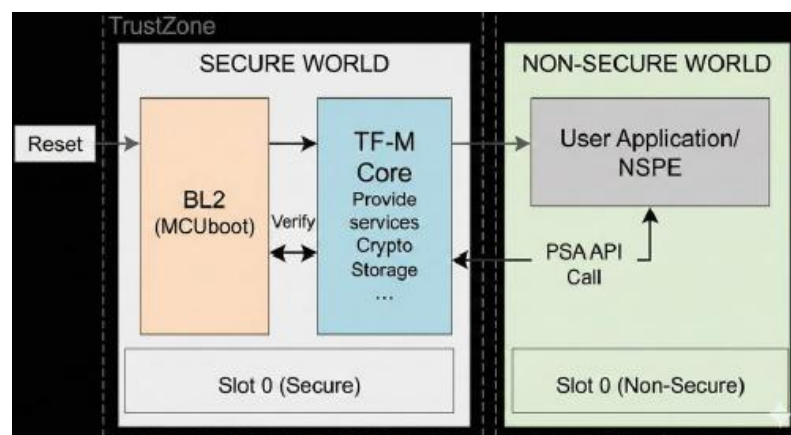


Figure 1: Isolation of secure and non-secure world

The non-secure world is the last destination and is trusted only because the previous stage said it was safe. It includes the main application for sensors, network stack and user's code. It cannot access the secure world directly. Instead, it communicates using PSA API calls and requests for secure services from the TF-M core. This isolation and separation of two worlds is important as it guarantees that even if some vulnerabilities are present in the sensor application code, critical operations and secrets remain protected in the secure world.

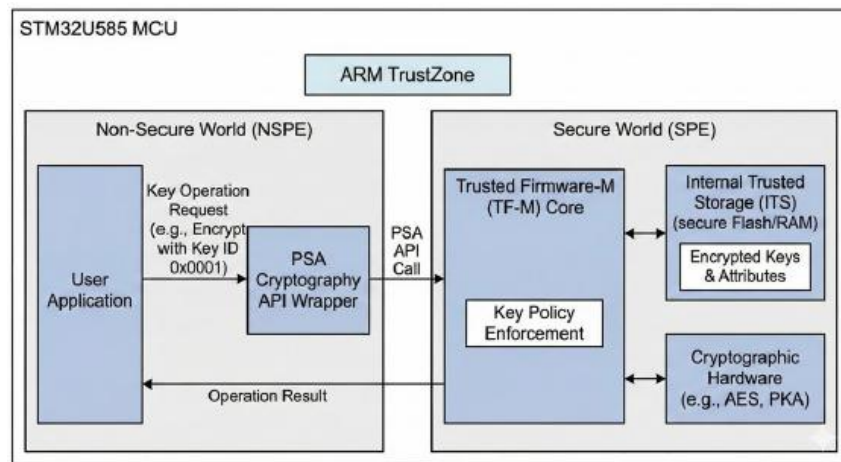


Figure 2: Communication between Secure and Non-secure world

### 5.3 Interrupt and peripheral isolation

STM32U585 MCUs also ensures that the peripherals can be configured as secure or non-secure with the use of securable I/O (*STM32U585xx Datasheet*). On-board Wi-fi and BLE modules in the kit will be operating in the non-secure domain and are vulnerable to attacks, while the cryptographic accelerators, random number generator and secure storage are included inside the secure world. Similarly, the interrupts generated by the secure peripherals are constrained to be within the secure world which prevents non-secure world to access them. This feature is essential for mitigating any cross-domain attacks.

## 6. Secure Data Storage

### 6.1 Hardware Unique Key (HUK)

HUK is the secret for each device that is fused during manufacturing (*B-U585I-IOT02A Discovery Kit — Zephyr Project Documentation*, n.d.). As the HUK cannot be directly read, secure AES (SAES) module uses it to derive session keys. Also, derived HUKs and boot hardware key (BHK) are utilized to encrypt the application keys. The SAES peripheral wraps and unwraps keys using these hardware secrets

and the derived keys are unique to each chip (*STM32U5: Enhanced secure key storage*). As the keys are always invisible to software or debuggers, the attack surface is reduced.

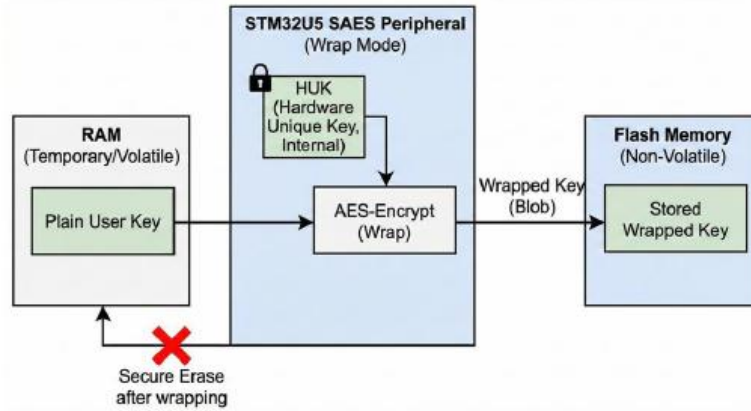


Figure 3: HUK transaction process

## 6.2 Secure flash and external memory

Internal flash memory and external QSPI/PSRAM memory are used for storing the collected sensor data. The MCU is capable of on-the-fly decryption of external memories (*B-U585I-IOT02A Discovery Kit — Zephyr Project Documentation*, n.d.). Encryption is enabled for all application code as well as data stored externally. Additionally, secure storage utilizes the HUK-derived keys for encrypting the sensor data, credentials and configuration parameters before writing them into the flash memory. As stated earlier, TF-M has protected storage service which makes sure that the confidentiality and integrity of the stored data is maintained the rollback is prevented. Additionally, sensitive credentials like Wi-Fi keys are stored in the internal trusted storage of the secure world.

## 7. Secure Communication Protocols

### 7.1 Choice of protocols

Transmission of sensor data by the system to the cloud service as well as receiving the firmware updates by the environmental sensor device is a crucial operation. For this environmental sensor device, Message Queuing Telemetry Transport (MQTT) over Transport Layer Security (TLS) will be used for sensor data and configuration commands. MQTT is widely used for IoT as it is a lightweight publish-subscribe protocol and it reduces overhead and additionally, supports persistent connections. As opposed to HTTP, MQTT is designed for high latency environments with low bandwidth and it also offers higher efficiency and better reliability than HTTP specially in IoT cases (Inc, n.d.).

Furthermore, Hypertext Transfer Protocol Secure (HTTPS) is chosen for firmware updates as it ensures the confidentiality and integrity of firmware images during download and provides authenticity if combined with the digital signatures.

## **7.2 Wi-Fi and BLE connectivity**

MXCHIP EMW3080 Wi-Fi module with 2.4 GHz bandwidth is included in the B-U585I-IOT02A kit along with a BLE module. As per ST's documentation, it supports Wi-Fi 802.11 b/g/n and BLE communication. Hence, for the design of the environmental sensor, Wi-Fi is used for sensor data transmission, firmware downloads and other communications requiring higher-bandwidth. BLE communication is employed to locally configure the system using a smartphone, which is protected by pairing and encryption. Both BLE and Wi-Fi interfaces sit on the non-secure world, however, all the sensitive operations like key handling occurs in the secure world.

## **8. Secure Wi-Fi FOTA (Firmware-Over-The-Air) update mechanism**

### **8.1 Overview of FOTA**

Device's firmware needs to be updated in order to add more features to device or patch any security vulnerabilities seen in the previous version of the firmware. This requires secure mechanism as there is risk of attackers providing with malicious code or even attempt to downgrade the firmware to exploit their vulnerabilities. The concept used in ST's secure firmware update involves separating the tasks of downloading, verifying and installing new firmware. This process can be achieved by using either a one slot (overwrite) or a two-slot (download then switch) mechanism (*Category:Secure Boot and Secure Firmware Update - Stm32mcu*, n.d.). In this design, two-slot strategy is selected as it reduces the downtime and enables rolling back to the previous version if there is failure in installation process.

### **8.2 Firmware update protocol**

The FOTA process can be designed to include following steps:

- Updated notification: The device can be set up to receive an update push notification with metadata including the newer firmware version and its size. The metadata can be used to prevent unauthorized different firmware version being installed on the device.
- Download via HTTPS: A secure server is used to host the firmware image and the device initializes the HTTPS connection, authenticates to use its certificate and downloads the encrypted update image in segments. It stores each segment in the non-active flash partition (second slot).
- Verification: After the device finishes downloading the update, the secure bootloader can verify the firmware image's digital signature using ECDSA which checks the firmware version number

and matches with the metadata. This prevents the risk of rollback. The secure bootloader then utilizes the device's root keys stored in HDP to verify its authenticity and additionally uses SHA-256 to compute a hash for verifying its integrity.

- Decryption: The image might be encrypted to ensure confidentiality. In this case, the bootloader works to decrypt it using AES-CTR-128 and HUK-derived keys.
- Installation: This verified and decrypted firmware image is then copied into the primary slot where the bootloader updates the metadata and sets a marker for update.
- Reset and boot: At reset, root of trust executes and goes through the process of validating the new firmware using the secure boot mechanism as described earlier. If it fails at any point, it reverts the device back to its previous firmware.
- Acknowledgement: If the new firmware is successfully installed, the device transmits an acknowledgement using MQTT to the update server. This server then records the successful installation of the new firmware version.

### **8.3 Anti-rollback**

As stated, the metadata of the update firmware image consists the firmware version to install. Anti-rollback protection prevents the attacker to install an older firmware version by refusing the images with version number lower than the current one. The secure bootloader can maintain a firmware version counter which can be stored in the protected area of flash and it cannot be modified by the non-secure code.

## **9. High-Level System Architecture**

### **9.1 Data acquisition**

Different environmental sensors like temperature sensor, humidity sensor, pressure sensor are connected to the microcontroller via different interfaces like I2C, SPI. Raw data is collected by the drivers running in the non-secure world and required pre-processing like filtering, calibration is done on the raw data. If the encryption is required the processed measurements are passed to the secure world.

### **9.2 Connection and communication**

In the non-secure world, the wi-fi module is controlled via SPI or UART interface and a lightweight TCP/IP stack also runs on non-secure memory. Sensor data is published on the cloud by this application using an MQTT client library. For establishing the connection, the non-secure code requests to provide the certificate and key via TF-M's secure storage API. Data is encrypted end-to-end between the device and the server. Additionally, commissioning is done by using BLE connectivity where a smartphone app

can be used to deliver Wi-Fi credentials or update configuration parameters. While pairing BLE, it uses passe-keys and protects the exchanges using an encryption.

### **9.3 Secure update and maintenance**

Downloading of update firmware images and storing them into the inactive slot is performed by non-secure code under the supervision of TF-M. The secure update process after downloading as described in the previous section is performed in the secure world. Secure bootloader provides all the services like cryptographic check and installation logic. If any update fails during the process, previous firmware version is automatically restored in the system. The device's firmware version can be accessed using MQTT or BLE to verify the update status.

### **9.4 Power management and tamper detection**

Longer battery life is enabled due to MCU's low power modes combined with its flexible options for supplying power. When the system is not sampling or transmitting the data, it just enters into the low-power mode to save the battery life. Since, the deployment of device can be in unsecured physical location, it is vulnerable to tampering. Physical intrusion is monitored via the active tamper detection lines. If any tampering even is detected, the secure world can erase the secrets or even lock the device.

## **10. Conclusion**

This work shows how a security architecture can be built for environmental sensor based on STMicroelectronics B-U585I-IOT02A Discovery kit. The kit has STM32U585AI MCU which provides numerous features like TrustZone, cryptographic accelerators, hardware RoT by which the device can fully leverage the design of secure boot, firm execution and robust data storage and protection. Secure firmware update to new version, protected storage service and other secure services are provided by the Trusted Firmware-M. The proposed system design uses MQTT for data transmission along with HTTPS for downloading firmware to update them. FOTA mechanism involving two-slot system was used for authentic update process and it also had the feature of anti-rollback and allows the device to recover from failure. Hardware security features were combined with the layered software architecture as well as the secure communication protocols. This allows the proposed design to achieve integrity, authenticity and confidentiality. These types of devices can be used as the basic framework for secure IoT sensors in multiple applications like smart buildings, industrial monitoring, environmental research and many others.

## References

- B-U585I-IOT02A | Product—STMicroelectronics*. (n.d.). Retrieved December 11, 2025, from <https://www.st.com/en/evaluation-tools/b-u585i-iot02a.html>
- B-U585I-IOT02A Discovery kit—Zephyr Project Documentation*. (n.d.). Retrieved December 11, 2025, from [https://docs.zephyrproject.org/latest/boards/st/b\\_u585i\\_iot02a/doc/index.html](https://docs.zephyrproject.org/latest/boards/st/b_u585i_iot02a/doc/index.html)
- Category:Secure boot and Secure firmware update—Stm32mcu*. (n.d.). Retrieved December 11, 2025, from [https://wiki.st.com/stm32mcu/wiki/Category:Secure\\_boot\\_and\\_Secure\\_firmware\\_update](https://wiki.st.com/stm32mcu/wiki/Category:Secure_boot_and_Secure_firmware_update)
- Inc, E. T. (n.d.). *MQTT vs HTTP: Ultimate IoT Protocol Comparison Guide*. Wwww.Emqx.Com. Retrieved December 12, 2025, from <https://www.emqx.com/en/blog/mqtt-vs-http>
- Moran, B., Tschofenig, H., Brown, D., & Meriac, M. (2021). *A Firmware Update Architecture for Internet of Things* (Request for Comments No. RFC 9019). Internet Engineering Task Force. <https://doi.org/10.17487/RFC9019>
- Security:STiRoT - stm32mcu*. (n.d.). Retrieved December 11, 2025, from <https://wiki.st.com/stm32mcu/wiki/Security:STiRoT>
- Understanding Physical Security Attacks | Exploiting Local Host and Physical Security Vulnerabilities | Pearson IT Certification*. (n.d.). Retrieved December 11, 2025, from <https://www.pearsonitcertification.com/articles/article.aspx?p=2952620&seqNum=3>
- What is Data at Rest | Security & Encryption Explained | Imperva. (n.d.). *Learning Center*. Retrieved December 11, 2025, from <https://www.imperva.com/learn/data-security/data-at-rest/>